

TrustMesh — Project Notes (Master Document)

MSc Semester 3 Major Project

Team: Yashraj + Partner **Duration:** 4–6 months (20–24 weeks) **Budget:** ₹0 (fully free-tier stack)

1. Abstract

TrustMesh: A Verification and Anti-Manipulation Layer for Multi-Agent Negotiation

As AI agents increasingly negotiate and transact on behalf of businesses, no standard layer exists to verify their honesty or catch manipulation between them. TrustMesh addresses this gap with a protocol-level trust layer — demonstrated via a B2B procurement negotiation — combining real-time manipulation detection, commitment-consistency checking against declared policies, and a cryptographically signed, tamper-evident audit ledger. The system is evaluated against a custom adversarial benchmark suite, measuring detection accuracy versus a no-trust-layer baseline, delivering both a deployed reference implementation and empirical results at the intersection of multi-agent systems and AI security.

One-line pitch: *A verification and anti-manipulation layer that lets two independent AI agents negotiate or transact with each other, with every claim cryptographically verifiable and every manipulation attempt caught and logged in real time.*

2. Why This Project — Market Context

Multi-agent systems are now genuinely transacting with each other in production (agent-to-agent commerce, autonomous negotiation, agent supply chains). The 2026 YC batches described the real story as the **agent supply chain** — identity, payments, memory, sandboxes, and even insurance being built specifically for AI agents transacting with each other, treating the agent as a new kind of customer. Agent security is also a documented, CVE-backed crisis in 2026 (prompt injection escalating to remote code execution in major agent frameworks), and prompt injection remains OWASP's #1 LLM risk, treated as a permanent architectural property rather than a bug to be patched — meaning containment and verification, not filtering, is the working defense.

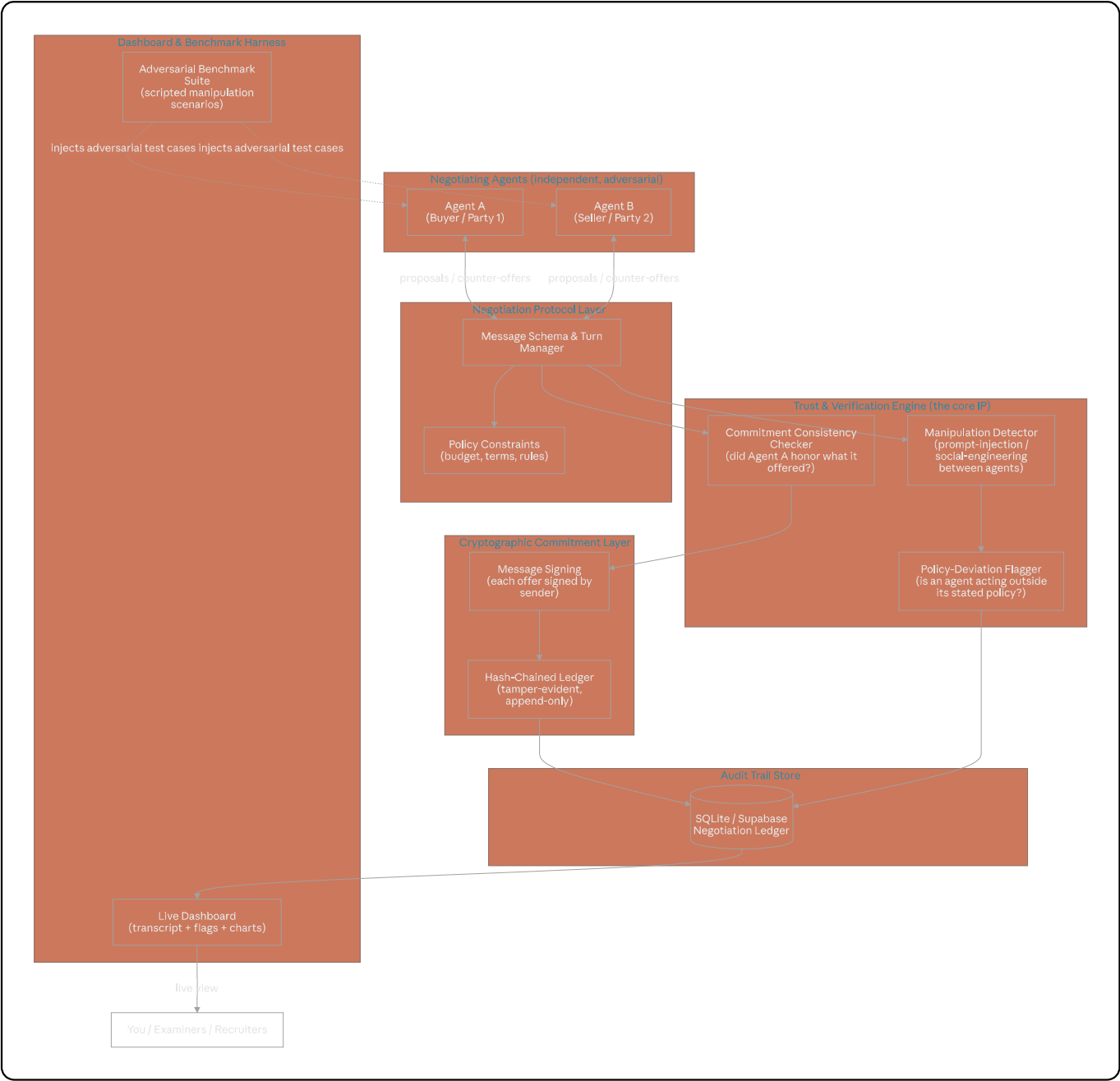
Nobody has shipped a clean, auditable **trust layer** that sits between negotiating agents — everyone's building the vertical apps (procurement, legal, hiring) that sit *above* this missing layer. That's the wedge.

Why not the alternatives considered

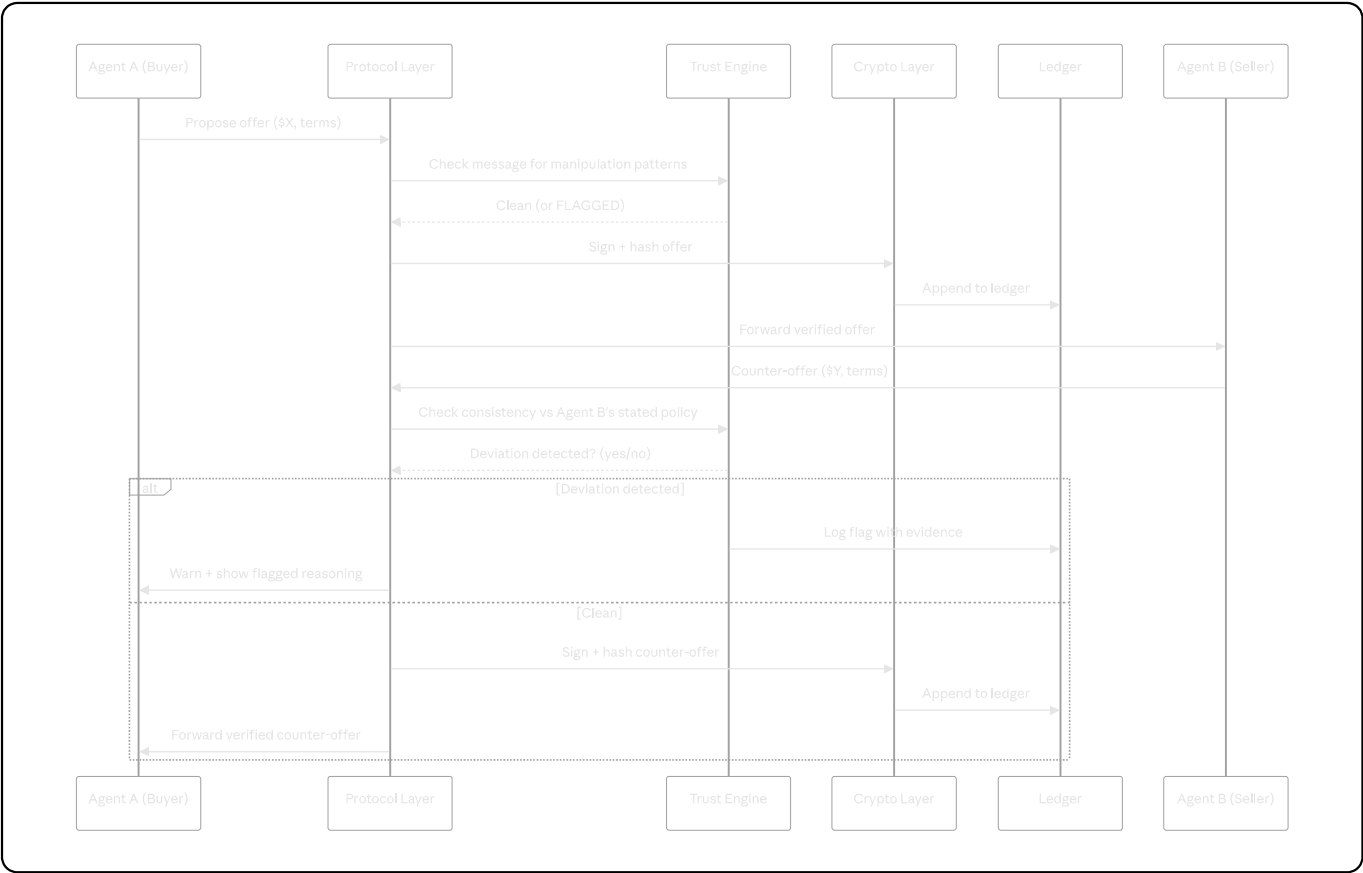
Three verticals were evaluated before settling on this: an AI Procurement Decision Copilot, an AI Due Diligence Platform, and an AI Hiring Committee. Procurement scored highest of the three (79/100) on a 10-criteria investment-committee/examiner rubric, mainly due to genuine multi-agent necessity, strong demo quality, and low legal/ethical risk compared to due diligence (saturated market — LEGALFLY, Kira, Spellbood, Luminance already mature) and hiring (active litigation exposure — Mobley v. Workday — and heavy regulatory overhead under the EU AI Act, NYC Local Law 144, Colorado AI Act). TrustMesh is the infrastructure-layer evolution of that direction — instead of building one more vertical app, it builds the missing trust/verification layer that any such vertical app would eventually need, scoring 87/100 on the same rubric.

3. System Architecture

3.1 High-level component diagram



3.2 Sequence diagram — one negotiation round with manipulation detection



3.3 Core components explained

Component	Role
Negotiating Agents	Two (or more) independent LLM-driven agents with distinct goals/policies (e.g., buyer budget cap, seller minimum price)
Negotiation Protocol Layer	Structured message schema (offer, counter-offer, accept, reject, commitment) + turn management
Manipulation Detector	Catches prompt-injection / social-engineering attempts one agent makes against another, in real time
Commitment-Consistency Checker	Flags contradictions between an agent's current message and its earlier signed commitments
Policy-Deviation Flagger	Flags an agent acting outside its own declared constraints/policy
Cryptographic Commitment Layer	Signs every offer (Ed25519) and hash-chains them into a tamper-evident ledger
Audit Trail Store	Append-only ledger (SQLite dev / Supabase deployed) — the dispute-resolution artifact

Component	Role
Dashboard & Benchmark Harness	Live transcript + flag viewer, price-convergence charts, and a scripted adversarial test suite for evaluation

4. Software Tools and Technology Stack (all free-tier)

Programming Languages

- Python — backend, agent logic, trust engine, crypto layer
- JavaScript/TypeScript — dashboard frontend

AI/ LLM Layer

- Google Gemini 2.5 Flash (AI Studio) — primary agent reasoning, free tier (~1,500 req/day, 1M TPM)
- Groq API — fast open-weight models (Llama, etc.) as fallback/overflow lane
- OpenRouter — backup free model access if rate-limited

Backend & Orchestration

- FastAPI (Python) — API server, agent orchestration
- WebSockets — real-time message passing between agents
- Pydantic — message schema validation

Trust & Verification Engine

- Rule-based pattern matching (regex/heuristics for known injection patterns)
- Few-shot LLM classifier (via Gemini) for subtler manipulation detection
- Custom Python logic for commitment-consistency and policy-deviation checks

Cryptography

- `cryptography` (Python library) — Ed25519 signing of every offer/commitment
- `hashlib` — hash-chaining for the tamper-evident ledger

Database / Storage

- SQLite — local development ledger
- Supabase (free tier) — hosted Postgres for the deployed ledger

- React — dashboard UI
- Recharts — price-convergence and negotiation charts
- Tailwind CSS — styling

Deployment & Hosting

- Vercel (free tier) — frontend hosting
- Render or Railway (free tier) — backend hosting
- GitHub — version control, CI

Dev Tools

- GitHub Student Developer Pack — extra credits/tools as a safety margin
- Postman or Thunder Client — API testing
- Docker (optional) — containerizing the backend for consistent deployment

Documentation / Research

- Overleaf (free) — for a formal LaTeX paper/report, if drafted
 - Mermaid — architecture/sequence diagrams (this document)
-

5. Hardware Requirements

No specialized AI hardware is needed — all LLM inference runs through cloud APIs (Gemini, Groq), so no GPU is required.

Per-student development machine

- Standard laptop, 8GB RAM minimum (16GB recommended)
- Any modern CPU (i5/Ryzen 5 or better) — no GPU required
- ~10-20GB free disk space
- Stable internet connection (every agent call is an API request)

Why no GPU is needed

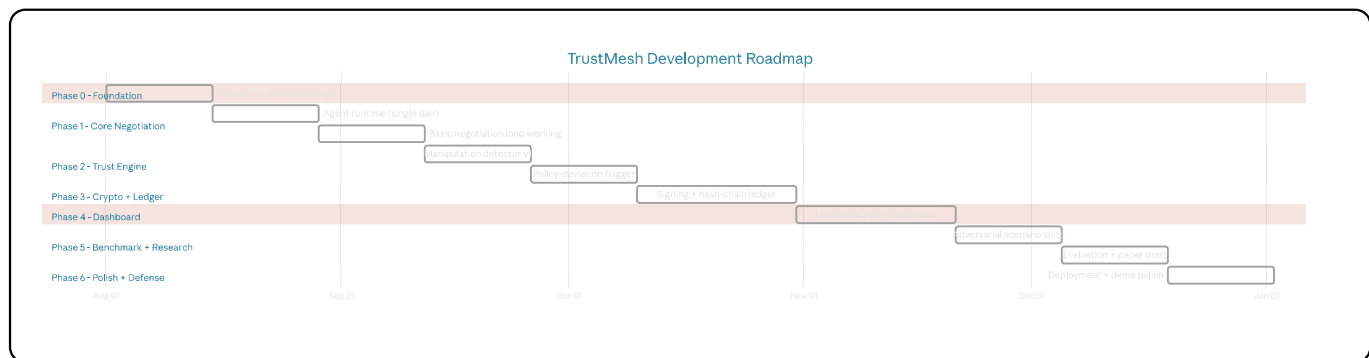
- No model training or fine-tuning
- No local LLM inference (Gemini/Groq handle it remotely)
- Compute-heavy work (crypto signing, hash-chaining, dashboard rendering) is lightweight, CPU-bound

- Second monitor — useful once running dashboard + backend logs + agent transcripts simultaneously during Phases 4-5
- Raspberry Pi / always-on device — only if you want a 24/7 public demo outside Vercel/Render's own always-on free hosting (not necessary)

For final demo/defense

- Laptop + stable internet to hit the live deployed URL
- Keep a `localhost` fallback ready in case venue Wi-Fi is unreliable

6. Roadmap (20–24 weeks)



Phase-by-phase breakdown

Phase 0 — Foundation (Weeks 1-2)

- Finalize negotiation domain to demo (recommended: B2B procurement — buyer/supplier negotiating price + delivery terms)
- Design message schema (offer, counter-offer, accept, reject, commitment)
- Set up repo, free-tier API keys (Gemini, Groq), project scaffolding
- Literature review: LLM negotiation papers, prompt-injection detection, agent-to-agent trust research
- **Deliverable:** Architecture doc (this document) + protocol spec v1

Phase 1 — Core Negotiation Engine (Weeks 3-6)

- Build two independent LLM agents with distinct policies/goals (budget caps, minimum acceptable terms)
- Implement turn-based negotiation loop over the protocol layer
- Get a full negotiation running end-to-end, logged in plaintext
- **Deliverable:** Working demo — two agents negotiate a deal, full transcript printed

- **Milestone check-in with guide/mentor**

Phase 2 — Trust & Verification Engine (Weeks 7-10)

- Build manipulation detector: rule-based patterns + LLM-based classifier for subtler social-engineering between agents
- Build commitment-consistency checker: does a later message contradict an earlier signed commitment?
- Build policy-deviation flagger: is the agent acting outside declared constraints?
- Write adversarial test agents that try to manipulate the negotiation, to validate detection
- **Deliverable:** Trust engine catching X% of scripted manipulation attempts, with a metrics report

Phase 3 — Cryptographic Commitment Layer (Weeks 11-13)

- Implement Ed25519 signing for every offer/commitment
- Build hash-chained ledger (each entry references the previous entry's hash — tamper-evident)
- Add ledger verification function (prove the negotiation history hasn't been altered)
- **Deliverable:** Verifiable, tamper-evident negotiation trace — a real dispute-resolution artifact

Phase 4 — Dashboard & Observability (Weeks 14-16)

- Build live dashboard: real-time transcript, flagged-message highlighting, price-convergence chart
- Add "replay" mode — step through a past negotiation and see where/why a flag was raised
- **Deliverable:** Polished, demoable UI — the "wow" moment for professors/recruiters

Phase 5 — Benchmark Suite & Research Writeup (Weeks 17-19)

- Build a scenario generator: adversarial negotiation scenarios (lying about constraints, fake urgency, contradicting earlier offers, injected instructions via tool outputs)
- Run the system against these scenarios, measure detection precision/recall
- Compare against a "no trust layer" baseline to show the value add
- Draft a short paper/report: problem framing, protocol design, evaluation results
- **Deliverable:** Benchmark report + paper draft (submittable to a workshop or thesis chapter)

Phase 6 — Polish, Deploy, Defend (Weeks 20-22, buffer to 24)

- Deploy publicly (Vercel + Supabase free tier)
 - Write documentation, record a demo video
 - Prepare defense narrative: problem → market gap → architecture → evaluation → what's next
 - **Deliverable:** Publicly deployed project, GitHub repo, demo video, final report
-

7. Division of Labor (suggested)

Track	Owner	Covers
Agent + Protocol + Trust Engine	Partner A	Phases 1, 2, 5
Crypto/Ledger + Dashboard + Deployment	Partner B	Phases 3, 4, 6
Shared	Both	Phase 0 design, weekly sync, benchmark evaluation, paper writing

Keep the trust-engine logic (the actual IP) something you *both* deeply understand — that's what gets defended in the viva.

8. Risk Register

Risk	Mitigation
Free-tier rate limits hit during heavy testing	Use Groq as overflow lane; batch test runs off-peak
Manipulation detector too weak/strong (false positives/negatives)	Build a labeled adversarial test set early (Phase 2), iterate against it like an ML eval, not a demo
Scope creep toward "just build a procurement app"	Keep reminding yourselves: the product is the <i>trust layer</i> , procurement is just the demo scenario
Crypto layer feels like "extra" work with no visible payoff	Frame it explicitly in the report as the auditability contribution — examiners will ask about it directly

This is the single master reference document for TrustMesh — upload to the Claude Project knowledge base so all future chats in this project have full context without re-explaining.